

{question_title}

Authored by: Arpit Raj • Institution: LNMIIT Jaipur • Date: July 12, 2026

Problem Overview & Intuition

The problem asks for the minimum number of elements needed to be added to an array *nums* such that the sum of the array elements becomes equal to a given *goal*. Each added element must satisfy the constraint that its absolute value is less than or equal to *limit* ($|element| \leq limit$).

To minimize the number of elements added, we should greedily choose the largest possible step size in each move, which is exactly equal to the *limit*. The total absolute difference that we need to bridge is given by $rem = |goal - sum|$. Dividing this remaining difference by the maximum allowed step size (*limit*) and taking the ceiling ensures we account for any fractional element needed to reach the target exactly.

Key Insight

Using a greedy approach is mathematically optimal here. Since there are no constraints on the number of elements but only on their maximum magnitude, filling up the remaining gap using the largest possible units (*limit*) guarantees the fewest steps.

Mathematical Formulation

Let the initial sum of the array be $S = \sum nums[i]$.

The total required delta or remainder to reach the target is:

$$rem = |goal - S|$$

The minimum number of elements required is the ceiling of the division of the remainder by the limit:

$$Ans = \lceil rem / limit \rceil$$

Complexity Analysis

TIME COMPLEXITY

$O(N)$

To compute the sum of the array using `std::accumulate`.

SPACE COMPLEXITY

$O(1)$

Only constant extra space is used for variables.

Formatted C++ Source Code

```
class Solution {
public:
    int minElements(vector<int>& nums, int limit, int goal) {

        int n = nums.size();
        long long sum = accumulate(nums.begin(), nums.end(), 0LL);
        int ans = 0;

        long long rem = abs(goal - sum);
        return ceil((1.0 * rem) / limit);

    }
};
```

Critical Implementation Details

- **Integer Overflow Prevention:** The sum of the elements in `nums` can easily exceed the standard 32-bit signed integer limits. Using `0LL` inside `std::accumulate` forces the accumulation loop to utilize 64-bit precision (`long long`), preventing runtime integer overflow.
- **Ceil Function with Floating Point Precision:** To execute ceiling division smoothly, the code converts `rem` to a floating-point type by multiplying it by `1.0` before performing the division by `limit`.